

PROJECTIONS

Kasozi Ismail

Math Club, Horizon International School Uganda

k.kasozi69@gmail.com

Imagine, if you will, that you are currently taking part in some elaborate series of escape room puzzles. After half a day of non-stop work, you finally make it to the final room. At the far end of the room, there is a terminal with ports on it. Just next to the terminal is a door with a keypad next to it. To your pleasant surprise, the instructions on how to acquire the code that unlocks the door are provided on a note left on the terminal. They read as follows: *“Node A1 outputs a repeating voltage waveform signal. This signal is composed of different frequencies, all whole numbers of hertz, and none of them is larger than twenty. The code that unlocks the door is composed of all the frequency values present in the signal, each written as two digits, arranged in descending order. In the only unlocked slot of the terminal drawers, you will find all the tools you need. Good luck”.*

This is the absolute worst. Of all things, it had to be this, you can’t help but think to yourself. It’s been a while since you last did any serious maths. You know that the solution has something to do with Fourier transforms, as you remember reading about them in your school days. *They’re used for splitting signals or something like that, right?* You open the drawer. In it, you find a raggedy old laptop, a USB oscilloscope and a bunch of cables.

You turn the laptop on and find that there are only two apps installed: a code editor and a basic chatbot. You immediately try asking the chatbot for help, but it refuses to help you directly and insists that it can only provide you with at most two explanations of whatever concept you need to learn. You type into the console: “Explain to me what the Fourier transform is”. Before you send the message, you have second thoughts. *Won’t it just dump a wall of technical jargon and math at me and call it a day? I can’t waste a query like this.* So you switch up the prompt: “Please provide me with a conceptual framework that will enable me to understand an explanation of how I could solve this problem”. *This should work; it’s technically not giving me a solution.* You hit enter, and after a few seconds, the chatbot spits out a wall of text. It goes:

It is beneficial to first consider some basic linear algebra. Vectors can be defined in many different ways, and the definition you get depends on who you ask. A physicist will tell you that a vector is a mathematical object that has both a magnitude and direction; a computer scientist will say that it’s an array of elements; and a mathematician, being a mathematician, will give the most abstruse answer: that vectors are elements of vector spaces that can be added together and scaled by numbers. An example of a vector would be

$$\mathbf{v} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}.$$

This vector is in $\mathbb{R}^2$, in two-dimensional space. To visualise this vector, a Cartesian plane with the traditional x and y axes would be used, with the first element corresponding to the vector’s position on the x-axis and the second element corresponding to the vector’s position on the y-axis. The point would be identified, and an arrow with its tail at the origin (0, 0) would be drawn.

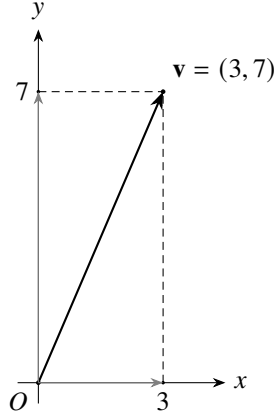


Figure 1: A vector in $\mathbb{R}^2$ represented via its coordinate projections onto the standard basis.

It is worth noting, though, that our decision to define the first element as the x-axis position and the second as the y-axis position is completely arbitrary. These two could be swapped in our rule, and it would still be a valid way of representing our vector. We see that the nature of the “space” these vectors live in depends on how we choose to define it. This should be kept in mind. We should consider what exactly is going on when we choose to identify our vector’s position in this way. Recall that every vector can be written as a linear combination of other vectors; therefore, our vector $\mathbf{v}$ is also

$$\mathbf{v} = \begin{bmatrix} 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 7 \end{bmatrix}.$$

Multiplying a vector by a number would “scale it”, meaning that it would stretch, squish or flip it in the vector space. The number that scales is fittingly called the scalar. For example, multiplying the vector $(3, 0)^T$ by the scalar 2 would stretch it to the vector $(6, 0)^T$, doubling its size. If that’s the case, then what stops each of these component vectors from being the result of such a scaling? Nothing! The vector $(3, 0)^T$, one of those that are combined to get vector $\mathbf{v}$, can be thought of as the result of scaling the vector $(1, 0)^T$ by 3. This vector, $(1, 0)^T$, is a “fundamental” vector that is scaled by all real numbers to produce all vectors along the x-axis. The same is true for $(0, 1)^T$ and the y-axis. These two fundamental vectors are called the basis vectors,

$$\hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \hat{\mathbf{y}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

So our vector $\mathbf{v}$ is really just the result of scaling $\hat{\mathbf{x}}$ by 3 and $\hat{\mathbf{y}}$ by 7 and adding the two together. In this way, we project a given vector along a basis vector. An n -dimensional space has n basis vectors, one for each of the dimensions. We say that for an n -dimensional space $\mathbb{R}^n$,

$$\mathbf{v} = \sum_{i=1}^n a_i \hat{\mathbf{e}}_i,$$

a weighted sum of basis vectors.

Now, another way of interpreting the vector $\mathbf{v}$, and the one most relevant to our problem, is how much each component of the vector (the x-component or the y-component) contributes to the overall vector (in this case, in terms of size). We can do this by getting the dot product

of our vector $\mathbf{v}$ with our $\hat{\mathbf{x}}$ basis vector, and with our $\hat{\mathbf{y}}$ basis vector.

$$\mathbf{v} \cdot \hat{\mathbf{x}} = \begin{bmatrix} 3 \\ 7 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 3, \quad \mathbf{v} \cdot \hat{\mathbf{y}} = \begin{bmatrix} 3 \\ 7 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 7.$$

The result of $\mathbf{v} \cdot \hat{\mathbf{x}}$ will give us a measure of how much of the vector is in the x-axis, and the result of $\mathbf{v} \cdot \hat{\mathbf{y}}$ in the y-axis (along the y-axis). In these operations, the basis vectors can be regarded as probes along their respective axes, searching for the strength of the vector along them. If the result of this probing for, say, $\hat{\mathbf{x}}$ were zero, it would mean that the vector has no strength along the x-direction. It is *orthogonal* to the basis vector. If we were to treat our vector as a signal, which it very well is, we have measured how much of the signal lies along each basis direction and know where most of the signal's energy is concentrated: the y-axis in this case. So, by thinking about how the vector is distributed in its 2-dimensional vector space, we can obtain a lot of insight into the components that make it up. Therefore, for a unit basis vector $\hat{\mathbf{e}}$, the scalar $\langle \mathbf{v}, \hat{\mathbf{e}} \rangle$ measures precisely how much of $\mathbf{v}$ lies along $\hat{\mathbf{e}}$. This is the component of the signal's energy concentrated in that direction.

To make the ideas underpinning this concrete, especially what exactly basis vectors are, we should think about what it takes for us to have a vector space with properly defined notions of geometry (lengths, angles, orthogonality). A vector space is merely a set of objects closed under addition and scalar multiplication, with these operations obeying a short list of natural rules (associativity, distributivity and the like). That is it. It doesn't need to have geometry, and yet, as we have seen, geometry is an important feature to attach to these spaces. The question then is how this is done. Well, it is accomplished via the inner product function. The inner product of two vectors (often denoted $\langle \mathbf{a}, \mathbf{b} \rangle$) is a scalar, and this scalar allows us to formally define geometric notions. A vector space with this inner product function is elevated to an inner product space.

In the finite-dimensional inner product space that is Euclidean space, this function is more commonly called the dot product. This is defined as

$$\mathbf{a} \cdot \mathbf{b} = \sum_{k=1}^n a_k b_k.$$

It isn't that hard to see how the dot product introduces geometry into a vector space, as it is used to determine the angles between vectors and hence establish orthogonality. Now basis vectors are just used as sort of reference vectors to build a coordinate system. It is convenient for the basis vectors of $\mathbb{R}^2$, $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$, to be defined as they were above. Any other set of vectors could have worked, as long as they spanned the entire space and were linearly independent (such that every vector is a unique combination of them), but we use these to keep things simple (more on that soon). It is along these basis vectors that we project our vector $\mathbf{v}$ and compute the dot product (inner product) as was already shown.

A nonzero result for a dot product computation between a basis vector and the vector we wish to analyse indicates that a portion of that vector can be found along that basis vector. A zero result, however, means that the vector is completely absent along that basis vector. This clean reading holds precisely because each probe is orthogonal to the directions it is not meant to measure, so no other component contaminates the result; orthogonal probes do not share any information and aren't entangled with each other in any way. It is very important that our basis vectors are orthogonal. Two vectors are orthogonal if and only if

$$\langle \mathbf{a}, \mathbf{b} \rangle = 0.$$

If they were not, then that value of 3 for the vector $\mathbf{v}$ along $\hat{\mathbf{x}}$ would also be shared with $\hat{\mathbf{y}}$. It would be much more difficult to truly isolate the vector's contribution along x . The basis vectors we chose also have the nice additional property of having a unit norm (which saves us the hassle of normalising projections to maintain consistency). They are therefore orthonormal (orthogonal with unit norm). This will not always be the case.

So, if this vector $\mathbf{v}$ were our signal and we wanted to know the strengths of the vector along the x and y directions, we would simply design “ x ” and “ y ” vector probes orthogonal to each other, get the inner product of each of the probes with the vector, which would neatly extract the strengths along these directions. Easy peasy!

You're taken aback by the quantity of output. Just then you notice a tiny screen on the terminal has lit up, and it's flickering red. *Please don't be what I think that is.* On checking, you realise that it is exactly what you think it is: a timer. There are ninety minutes left on the clock. There's no time to waste, and you immediately get to reading. You soon realise that the concepts in the text are not that hard to grasp. *So if I regarded a vector as a signal and I wanted to know its strength in different directions, I would compute the inner product of the vector and the basis vector of that direction.* In this case, it seems that the strength of the signal was how much it scaled a basis vector in a particular direction. *I'm looking for frequencies though, so I don't know how this helps.* You're stumped.

Since you can't yet solve the main problem, you decide to tackle a simpler challenge. The voltage signal is analogue and cannot be processed by a computer in this state. You need to digitise it by taking discrete measurements at fixed time intervals. Luckily, you know how to use the oscilloscope because the previous room involved using one, albeit in a completely different context. One of the wires in the drawer was a probe with that distinct round connector on one end and a metal tip on the other. You plug the probe into the device, and a third icon pops up on the desktop. *It must have been hidden to trigger only when I get this far, so at least I know I'm doing something right.* You open the new app and get a black screen ruled into squares by a faint grid, one flat green line lying dead across the middle, and a panel of controls with names like Volts/div, Time/div, Trigger and Capture on the side. You snap the ground clip onto a node labelled “GND” on the terminal and touch the probe tip to the node labelled “A1”. Something that looks like a signal starts dancing around on the screen. You proceed to set the trigger, and a stable signal appears on the screen.

You play around with the Volts/div dial until the wave grows to fit the screen. Then you adjust Time/div to set the time each square represents. You stretch and squeeze the signal with this knob until the pattern repeats four times across the screen. Now comes the main part, digitising the signal. You are asked to set two parameters before this: the number of samples you want to record and the rate at which you want to record them. The oscilloscope will take a snapshot of the voltage at an instant in time by reading it, writing it to memory, waiting for a fixed amount of time and then reading and writing it again. If these snapshots were spaced too far apart in time, you would fail to record the fine, fast-moving parts of the signal that oscillate wildly quickly and produce a poor approximation. The recording window is fixed to what is on the screen, so cramming in more snapshots means taking them closer together in time. You set the number of snapshots to the largest one on offer, 16384 discrete voltage measurements, and let the scope fix the rate to match. You hit record, the scope captures exactly the window on the screen, and a CSV file appears on your desktop.

You open the file in your code editor. There are two columns. The first is time, whose values are climbing in perfectly even steps as you go down the file. The second is voltage, and there are 16384 rows of it. This is the signal digitised, an array of 16384 elements. *An array of elements. This is what a computer scientist calls a vector!* This list is a vector. Not a 2D one like in the

example, but a 16384-dimensional one. *So what are the basis vectors here?* There should be as many basis vectors as there are dimensions, and if you want the clean probing the chatbot described, they should all be orthogonal to each other. *Hmmmmmm. . . so an array with a “one” at the start followed by 16383 zeros would be one basis vector. Then if I move the one, record the new array and repeat this 16383 times, I’d have all my basis vectors!*

$$\begin{aligned}\hat{\mathbf{e}}_1 &= [1, 0, 0, \dots, 0] \\ \hat{\mathbf{e}}_2 &= [0, 1, 0, \dots, 0] \\ &\vdots \\ \hat{\mathbf{e}}_{16384} &= [0, 0, 0, \dots, 1].\end{aligned}$$

These are my probes. All I have to do now to get the strengths is to compute the inner products of the signal and each of them.

You create a new Python file and quickly write up a few lines of code: load the voltage column from the CSV, take the inner product with each basis vector and print the strengths to the terminal. There is a problem, though. You can feel it; you’ve overlooked something. You glance at the timer, *fifty-two minutes*. If there’s something wrong, you’ll fix it in the next iteration of your solution; there’s no time to hesitate. You run the code and read the output. *There’s no way*. It feels as if you’ve done no work at all. The strength of the signal along $\hat{\mathbf{e}}_1$ is just the first entry in the signal array, and that along $\hat{\mathbf{e}}_2$ is the second entry. This pattern repeats for the rest of the results. *How could I have been so stupid! If I’d just actually thought about it for just one second. Taking the inner product with $\hat{\mathbf{e}}_2$ multiplies every component of the signal by zero except the second one, and then sums. The sum should obviously be just that one number.*

Disappointed, you glance at the timer. *Forty-five minutes left*. It seems hopeless. There has to be something that you’re missing. When you compute the inner product of this signal and a basis vector probe, the result tells you by how much the signal scales the basis vector: its “strength” along that direction. *What does strength mean anyway? The chatbot used this word rather loosely. It would be nice if the strength just meant how much of a frequency is in the signal, as that would be enough to tell you that that frequency exists in the signal in the first place.* Your eyes widen as another series of thoughts stream through your mind. *The most natural way to model periodic phenomena like this voltage signal is with the trigonometric functions. The amplitudes of these trig functions are the coefficients that scale them. There is a positive correlation between them and the strength of a signal, so they can serve as a measure of signal strength. So, if I replaced my simple basis vector probe with one of these trig functions at a frequency I wanted to query, and projected the vector signal along that probe, then. . .* You slam your hand on the terminal in a fit of excitement. You can feel your heart pounding in your chest. This is the closest you’ve felt to solving the problem all day.

But something feels wrong. It feels like you’re overlooking something again. This time you slow down and try to think deeply about it. *Using trig functions as probes means that I am treating them as my basis vectors. What kind of space would I create by doing this? A function as a basis vector. What would be the size of this vector? A function has a value at every single one of the infinitely many instants in its domain, one coordinate per instant. Doesn’t that mean that the space would be infinite-dimensional? So far, all the maths you’ve been using is for finite-dimensional Euclidean space. The space that would be defined by these functions would have an infinite number of dimensions. What does it even mean to take the dot product of a function? Once again, you’re stumped. I guess it’s time to use up my last query. This is uncharted territory.* You navigate to the chat window. *Please don’t spit out something too difficult to comprehend; I only have thirty minutes.* You type into the console all of your progress thus far and end with a simple query: “I know that you will not directly provide me with a solution to my problem, so at

least expound on the theory that I would need to move beyond my current sticking point”. You can only pray that this is the last piece of the puzzle, for you’d be on your own after this final reply. The chatbot goes:

Signals are usually the products of some phenomena through time, and these phenomena can be modelled by functions (that assign a value at every instant of time). So when we decide to take measurements of some signal-generating physical process, like a heartbeat, we are essentially sampling a function that describes the temporal behaviour of that process and storing its value at finitely many instants in a list. That list is a vector. Even more interestingly, functions themselves obey the same algebraic rules as vectors, as a sum of two functions f and g is defined pointwise as

$$(f + g)(t) = f(t) + g(t),$$

and scaling by a constant behaves identically. In this sense, functions are infinite-dimensional vectors.

We already established that functions live in an infinite-dimensional space. If we could project our signal into this space, along basis vectors that hold information about frequency, then we could find the coordinates of this signal, what frequencies it’s composed of and how strongly these frequencies contribute to the signal. The problem is that the operations we have used so far and the basis vectors only work for the finite-dimensional Euclidean space. We need to first extend this space into an infinite-dimensional one. Before we start looking for appropriate basis vectors, we first need to turn the discrete dot product into a continuous analogue, such that we can operate on functions instead of just finite arrays. This key change is precisely what will enable the extension to a proper function space.

The continuous version of discrete summation is integration. Therefore, the dot product becomes

$$\langle f, g \rangle = \int f(t) g(t) dt,$$

which is the inner product for vector spaces defined in infinite-dimensional space. The Euclidean inner product is a finite sampling of “pointwise correlation”. In contrast, this function-space inner product is the limit of that correlation as the sampling becomes infinitely fine (think of Riemann sums). This definition is, however, incomplete. The integral operation needs limits. This choice of limits, as we’ll see later, is vital to the definition of the space that we seek to construct. Changes in this interval alter the very geometry of the inner product space (the lengths, angles and even which vectors are orthogonal to one another).

Not every function can have an inner product defined over an interval. Consider the inner product of a function with itself,

$$\langle f, f \rangle = \int_{-\pi}^{\pi} f(t)^2 dt.$$

Squaring the function before integrating in this way has some nice physical interpretations. In signal processing theory, this squared function yields a quantity that often corresponds to the energy of a signal. This makes the squared norm a natural measure of signal strength. If this integral diverges, the function has infinite length, and no sensible geometry can be built around it (as it would imply infinite energy in our case). So we filter out such functions and

admit exactly the functions of finite energy, those satisfying

$$\int_{-\pi}^{\pi} |f(t)|^2 dt < \infty.$$

This family of functions, over the interval $[-\pi, \pi]$, is called $L^2([-\pi, \pi])$: the square-integrable functions on that interval. This infinite-dimensional space, with its notions of lengths and angles supplied by the integral inner product function, is called a **Hilbert space**, after the legendary mathematician David Hilbert. (Strictly speaking, the name also requires the space to be complete, meaning that limits of well-behaved sequences of functions do not escape the space.) It is into this space that we will project our signal. Of course, the computer will never manipulate the infinite-dimensional function directly. Instead, it will manipulate a finite collection of samples that approximates it.

We now need to choose appropriate basis vectors. The simplest possible function that possesses a single frequency and nothing else is a sinusoid: $\cos(nt)$ or $\sin(nt)$. To probe our signal for a particular frequency n , we can use such sinusoids, with the inner product reporting how much of the signal is contained in that frequency. So, these sinusoids can be regarded as our basis vectors. More concretely, consider

$$\langle \cos(nt), \cos(mt) \rangle = \int_{-\pi}^{\pi} \cos(nt) \cos(mt) dt,$$

where we treat $\cos(nt)$ as the signal we're trying to probe and $\cos(mt)$ as the basis vector probe. Using the expansion

$$\cos(nt) \cos(mt) = \frac{1}{2}(\cos((n-m)t) + \cos((n+m)t)),$$

the integrand becomes a sum of cosine functions. If $n \neq m$, each of these cosines completes an integer number of oscillations over $[-\pi, \pi]$, so the positive and negative contributions cancel exactly, causing the integral to vanish. These two functions are therefore orthogonal to each other, and this tells us that the frequency m is absent in our signal $\cos(nt)$, which makes sense when you think about it. If $n = m$ (with $n \geq 1$), however, the first term becomes $\cos(0) = 1$, and the integral comes out to π . The one exception is $n = m = 0$: the integrand is then just 1, and the integral comes out to 2π instead, so the constant probe is a special case worth remembering later. (The full calculations of these cases are left to the reader.) The sines behave identically among themselves, and a sine is always orthogonal to a cosine over the interval, whatever their frequencies. We can see how our inner product function for Hilbert space nicely mirrors the dot product for Euclidean space. So, the sinusoids can be used as basis vectors. (That this family is rich enough to build every finite-energy signal on the interval, in other words that it spans the whole space, is a deep fact that we will also take on faith.)

Although $\cos(nt)$ and $\sin(nt)$ are two different basis vectors (they are orthogonal), their inner products with a signal,

$$\langle f, \sin(nt) \rangle \quad \text{and} \quad \langle f, \cos(nt) \rangle,$$

would tell us a similar thing: that the frequency n is present in our signal. It is tempting to probe with just one of the pair, but that would be unreliable, as a component that happens to be a pure sine registers as zero against the cosine probe, since the two are orthogonal, and would slip by completely undetected. Both projections, $\langle f, \cos(nt) \rangle$ and $\langle f, \sin(nt) \rangle$, are

therefore needed at every frequency. We can neatly bundle these two, however. To do this, we can use the complex exponential,

$$e^{int} = \cos(nt) + i \sin(nt),$$

so that a single projection,

$$c_n = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(t) e^{-int} dt,$$

performs both probings at once. The inner product of complex-valued functions conjugates the second argument, $\overline{e^{int}} = e^{-int}$. This is necessary because without conjugation, the inner product of a complex function with itself could yield a negative or even complex number, which would be nonsensical as a squared length. Conjugation guarantees that $\langle f, f \rangle = \int |f(t)|^2 dt \geq 0$. Expanding the conjugate $e^{-int} = \cos(nt) - i \sin(nt)$, the projection separates into:

$$c_n = \frac{1}{2} \left(\frac{1}{\pi} \int_{-\pi}^{\pi} f(t) \cos(nt) dt \right) - \frac{i}{2} \left(\frac{1}{\pi} \int_{-\pi}^{\pi} f(t) \sin(nt) dt \right) = \frac{a_n - i b_n}{2}.$$

The real part carries the cosine projection a_n and the imaginary part carries the sine projection b_n , each halved. It is as though, at each frequency n , we carve a small $\mathbb{R}^2$ slice out of the infinite-dimensional space and project our signal into that plane. The magnitude of the resulting vector in the complex plane,

$$|c_n| = \frac{1}{2} \sqrt{a_n^2 + b_n^2},$$

combines both readings into half the overall amplitude of the signal at frequency n , regardless of phase. In essence, this magnitude is the strength of the signal at frequency n .

By the time you're done reading, there's only ten minutes left, but it doesn't matter anymore, for you already know what to do. The first thing you realise is that you don't care what the "strength" of a signal along a particular frequency is, only that the frequency exists in the first place. *So that's how I define the sinusoids as basis vectors. Now all I have to do is project my signal along them and check the inner product at each frequency. I need to define a complex exponential probe array with the same length as my signal vector. The chatbot's e^{-int} was defined over $[-\pi, \pi]$, with n counting oscillations in radians per unit time; my time column is in seconds and my frequencies are in cycles per second, so my probe should be $e^{-2\pi i n t}$, one that completes exactly n full cycles every second. I'll populate it by computing its value at every timestep in the CSV. So I still end up with a finite-dimensional vector, huh?*

You can't believe it. You rushed into things again. *So I still need to compute a dot product; it's just that this time I defined a different basis vector. I'll pass my complex probe for each frequency $0 \leq n \leq 20$ against the signal, and if the magnitude of the result is clearly above zero, then I'll append that frequency n to a separate array.* It's not all bad, though, as the exposition genuinely helped you better understand what was going on and gave you the complex exponential probe as a neat time-saving tool. Now that you have a clear plan, you write up a quick little script. There are things that you realise you need to pay attention to: the scaling factors for normalisation and the final complex number magnitudes. You come up with:

```
1 import numpy as np
2 import csv
3
4 t = []
5 f = []
```



```
6 with open('signal.csv') as file:
7     rows = csv.reader(file)
8     next(rows)
9     for row in rows:
10         t.append(float(row[0]))
11         f.append(float(row[1]))
12
13 t = np.array(t)
14 f = np.array(f)
15 N = len(f)
16
17 comps = []
18 mags = []
19 for n in range(0,21):
20     probe = np.exp(-2j * np.pi * n * t)
21     if n == 0:
22         c = np.dot(f, probe) / N # constant probe, no double counts
23     else:
24         c = np.dot(f, probe) * 2 / N
25     m = np.sqrt(c.real**2 + c.imag**2)
26     comps.append(c)
27     mags.append(m)
28     print(c)
29
30 freqs = []
31 for n in range(0,21):
32     if mags[n] > 0.1:
33         freqs.append(n)
34
35 freqs.sort(reverse=True)
36 print(freqs)
```

Now there's only one thing left to do. You run your code. It spits out an output almost immediately (thank you, NumPy). It turns out that there were only five frequencies present in the signal: [20, 17, 12, 8, 3]. Sitting right under them is the assembled code itself, "2017120803". A wave of relief washes over you, but it's soon followed by stifling anxiety. *What's wrong with me? I'm not even sure that this is the right code, and I was starting to feel like I was done.* Maybe deep down, you know that the problem is truly solved. You walk over to the door and type the code in. The keypad flashes red; the code is wrong. *No way.* You're holding back tears at this point, but then you glance at the keypad's little display. A stray five had somehow made it into your entry, right in the middle of the code; a slip of the finger. You compose yourself and carefully type the code in again. It beeps twice, and you hear a thunk on the door. You turn the handle and the door swings wide open.

SOME WORDS FROM THE AUTHOR

I first encountered Fourier Transforms when I was reading a paper titled "DEMO: Decoupled Momentum Optimization" from the research lab, Nous Research. In it, the Discrete Cosine Transform was used to compress tensors carrying momentum information before they were shared to different GPUs during the training of neural nets over distributed compute clusters. By reducing the size of the information that had to be transmitted, the authors reduced the bandwidth required to achieve model convergence and allowed for things like geographically distributed compute.

In this paper, the DCT was used to extract and select the top-k frequencies present in the tensor (which was taken to be a signal that could be compressed). Since these top frequencies would contain most of the information in the momentum tensor, the rest of the frequencies could be discarded, and the signal could be reconstructed using just those top frequencies. It was a really clever method with an amazing result. I spent the next two months trying to learn about this transform. I ended up writing a simple implementation of the Discrete Fourier Transform on a toy signal I crafted.

Seeing math on paper turn into actually working code is such a satisfying experience. Some of the components of this story mirror my own thought process as I was learning. Most of the texts that I consumed were really hard to understand at first, and I had to consult multiple sources (lecture notes from Gilbert Strang on OCW, an amazing playlist on YouTube by Professor Brad Osgood and Claude Opus) to help me better understand these ideas. When I wrote this essay, I wanted to write something that I would have loved to have had when I was just starting out. I hope that it does a good job of capturing the joy of discovery and imparting a good intuition for why Fourier transforms are necessary.

I'm currently working on applying the DCT to compressing part of the neural network training process, such that I can reduce the computational requirements to train or run certain networks. Thank you so much for reading my story, and I hope that it taught you something new.